Programação Concorrente

Processos

Professor: **Jeremias Moreira Gomes**
E-mail: jeremias.gomes@idp.edu.br

Introdução

Introdução

- O conceito mais central em qualquer SO é o de **Processo**
  - Uma das abstrações mais antigas relacionadas a SO
  - Permitem operações (pseudo) concorrentes,

    transformando uma CPU em múltiplas CPUs virtuais

Introdução

- **Uma abstração de um programa em execução**
  - É apenas uma instância de UM programa em execução
    - Cada processo tem sua própria CPU virtual
    - A CPU real troca a todo momento de processo em processo
      - Esse mecanismo (trocas rápidas) é chamado de **multiprogramação**
  - Um programa por si só não é um processo
    - Um programa é uma entidade passiva
    - Um processo é uma entidade ativa

Introdução

- Computadores realizam **várias tarefas ao mesmo tempo**.
  - Exemplo: servidor web (checa *cache*, acessa disco em caso de *miss*, e , devolve informação)
  - Acessos a **disco demoram muito mais** tempo que outros tipos de processamento
  - Uma **CPU poderia fazer outras requisições** a diferentes dispositivos antes mesmo que o primeiro terminasse
  - Modelar essa possibilidade (concorrência) é algo desejável

Introdução

- A ideia de processos auxilia essa possibilidade
- O que acontece quando o SO é inicializado?
  - Alguns processos são “secretamente” iniciados
    - Sincronização de emails
    - Antivírus
    - Etc
- Além disso, outros processos podem ser executados
  - Impressão, cópia de arquivos, etc

Introdução

- Todas essas atividades precisam ser gerenciadas por um sistema (multiprogramado) que dê suporte a múltiplos processos
- É comum em um sistema multiprogramado a CPU mudar (chavear) entre processos rapidamente
  - Cada processo executa por poucos milissegundos na CPU
- A CPU (tradicional) executa somente um processo por vez
  - Em 1 segundo, vários processos podem ser executados
    - Ilusão de paralelismo (pseudo paralelismo)

Introdução

- Controlar múltiplas atividades em paralelo é difícil para pessoas
- Projetistas de SO desenvolveram um modelo conceitual para lidar com esse “paralelismo”
  - Torná-lo mais fácil de lidar

Introdução

- Para conhecer esse mecanismo, vamos explorar os seguintes tópicos:
  - Modelo de Processos
  - Criação de Processos
  - Término de Processos
  - Hierarquia de Processos
  - Estados de Processos
  - Implementação de Processos
  - Modelagem da Multiprogramação

Modelo de Processos

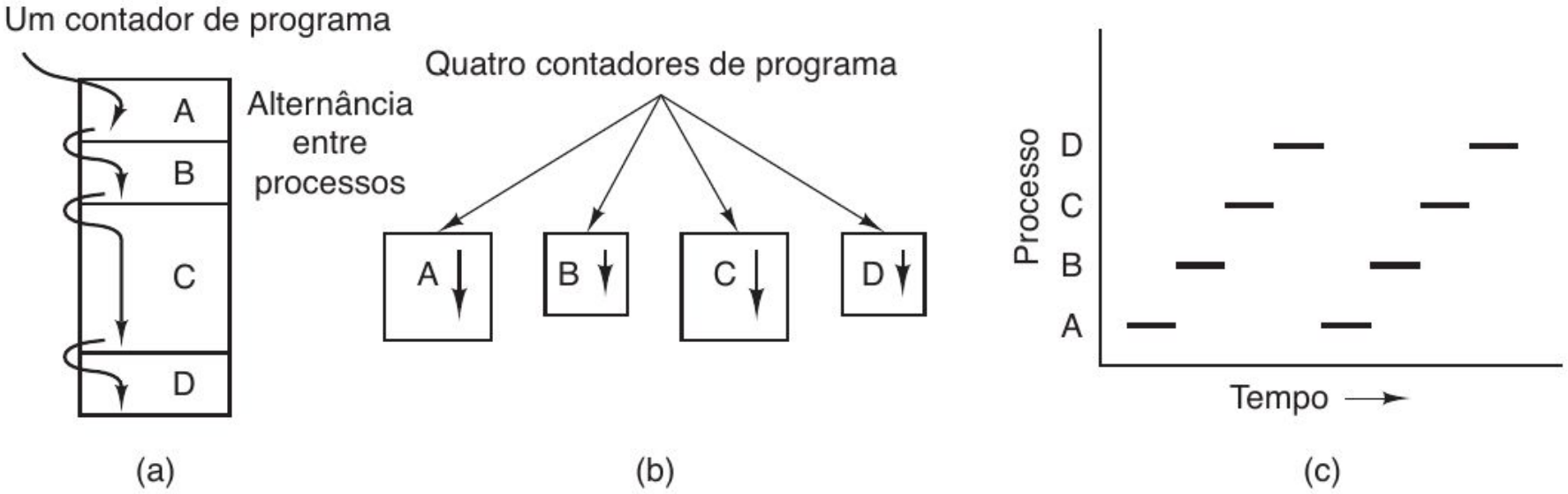
- No modelo de processos (que iremos estudar e utilizar), **todos os softwares** executáveis são **organizados como processos sequenciais**
- Um processo é:
  - **Uma instância de um programa em execução**
  - Valores atuais do contador de programa
  - Registradores
  - Variáveis

Modelo de Processos

- Cada processo possui sua própria CPU virtual
  a. A CPU real troca a todo momento de processo em processo
  b. Essas trocas (rápidas) são chamadas de **multiprogramação**
- O fluxo de controle de programas na memória pode ser visualizado de três formas:
  a. Pelo Contador de Programa (PC) real
  b. Cada programa com seu próprio Contador de Programa (PC)
  c. Pelo tempo de execução na CPU

Modelo de Processos

- Tipos de visualização a partir do fluxo de controle:



Modelo de Processos

- **O chaveamento** entre processos **não é uniforme**
  - **Processos não devem** ser programados com suposições sobre tempo
    - Exemplo: atraso em áudio e vídeo
  - Se há essa necessidade (tempo real), medidas especiais precisam ser tomadas para assegurar a ocorrência
  - Normalmente, a grande maioria dos processos não é afetado pela multiprogramação

Modelo de Processos

- Recursos utilizado por um processo:
  - **Memória**: um processo precisa de memória para armazenar instruções e dados
  - **CPU**: um processo precisa de posse da CPU para executar instruções
  - **Dispositivos**: um processo precisa/pode realizar alguma tarefa de entrada e saída em dispositivos (teclado, disco, etc)
  - **Arquivos**: um processo precisa gravar ou recuperar informações

Modelo de Processos

- A diferença entre processo e programa é essencialmente crucial
  - Analogia de um **pai preparando um bolo de aniversário**
    - Receita de bolo - programa (algoritmo)
    - Ingredientes - dados de entrada
    - Cozinheiro - processador
    - Processo:
      - Leitura da receita
      - Busca dos ingredientes
      - Preparo do bolo

Modelo de Processos

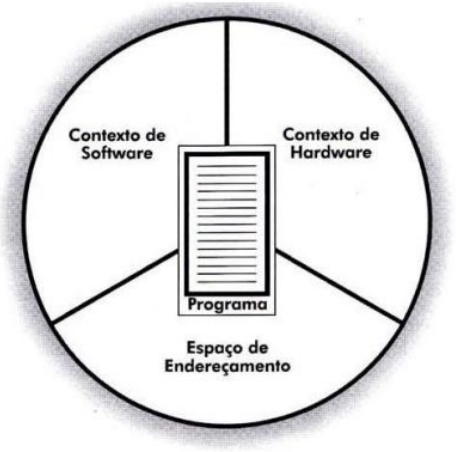
- A diferença entre processo e programa é essencialmente crucial.
    - Analogia de um **pai preparando um bolo de aniversário**.
        - Imagine que o **filho aparece** de repente, após s**er picado** por uma abelha.
            - O **pai para o que estava fazendo** e passa a aplicar os primeiros socorros
                - Essa mudança de atividades é a troca de um processo por outro, **por questões de prioridade**
            - Após o término do processo prioritário, o preparo do bolo é continuado de onde parou

Modelo de Processos

- **Processo é uma atividade** (ideia fundamental)
  - Tem programa, entrada, saída e estado
  - **Um único processador pode ser compartilhado** entre processos
- Um programa é algo que pode ser armazenado em disco sem fazer nada
- Se um programa é ou está **executando duas vezes**, eles **contam como processos diferentes**
  - Imprimir um arquivo duas vezes ou abrir um editor duas vezes
  - Ambos os casos, esses programas constituirão processos distintos

Modelo de Processos

- Um processo possui diversas informaçẽos a seu respeito
- Essas informações recebem o nome de **Contexto**
- O contexto de um processo é formado por três elementos básicos:
  - **Contexto de Hardware**
  - **Contexto de Software**
  - **Espaço de Endereçamento**

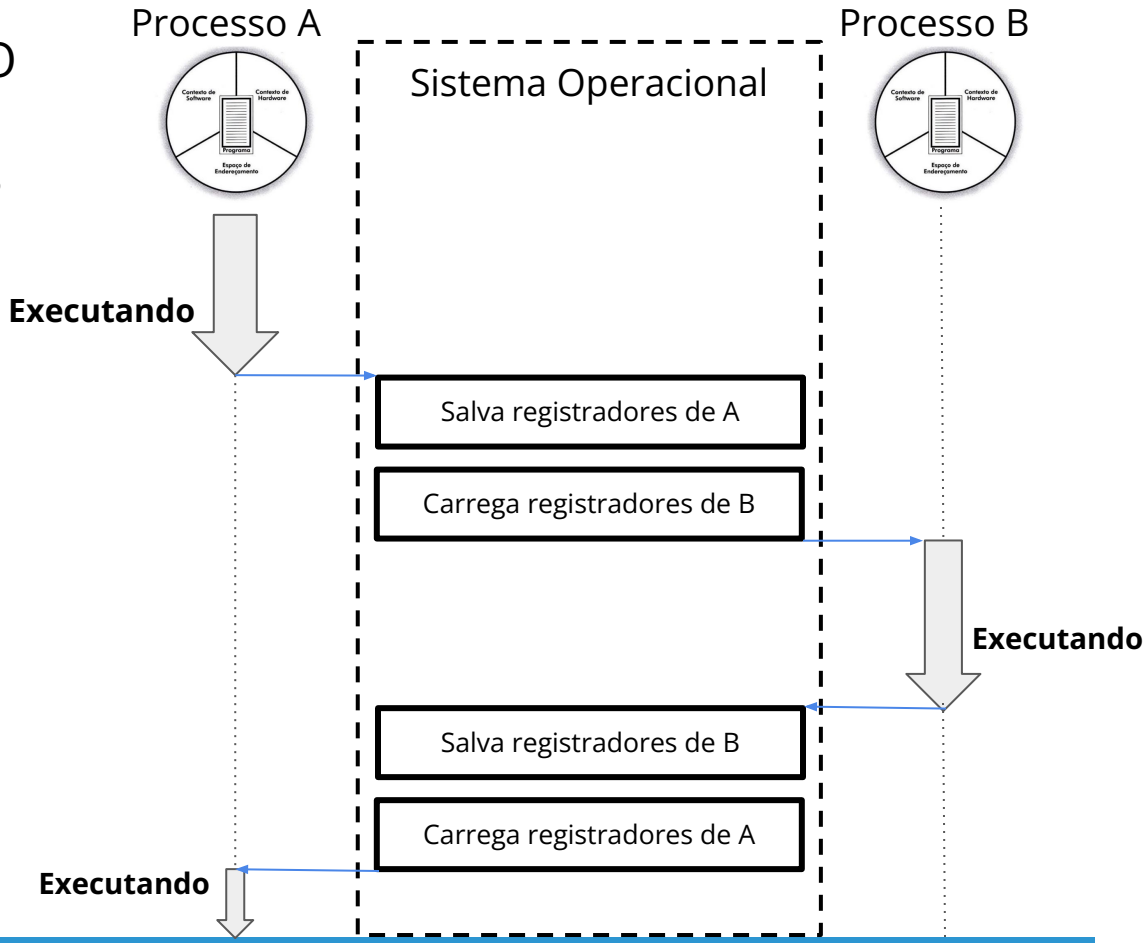


Modelo de Processos - Contexto de Hardware

- Constitui-se do **conteúdo dos registradores** (PC, SP, etc)
- O contexto de um processo em execução está nos registradores do processador
- O contexto de hardware é fundamental para a **implementação de sistemas de tempo compartilhado**, de forma a permitir o revezamento de utilização do processador
- A troca de um processo por outro na CPU é denominada **Troca de Contexto**

Modelo de Processos - Contexto de Hardware

- Na troca de contexto, o SO armazena o conteúdo dos registradores da CPU



Modelo de Processos - Contexto de Software

- Especifica características do processo que influenciam na execução de um programa
- Composto de três informações:
  - **Identificação**: PID (*Process Identification*) e UID (*User Identification*)
  - **Quotas**: Limites de cada recurso do sistema para o processo
  - **Privilégios**: O que o processo pode ou não pode fazer
    - Em relação ao Sistema Operacional
    - Em relação a outros processos

Modelo de Processos - Contexto de Software

```
/bin/bash
                                                    /bin/bash 103x23
[5654] j3r3mias@tardis:tmp > ps -eao uid,uname,pid,ppid,pmem,comm
  UID USER       PID  PPID %MEM COMMAND
    0 root         1     0  0.1 systemd
    0 root         2     0  0.0 kthreadd
    0 root         4     2  0.0 kworker/0:0H
    0 root         6     2  0.0 mm_percpu_wq
    0 root         7     2  0.0 ksoftirqd/0
    0 root         8     2  0.0 rcu_sched
    0 root         9     2  0.0 rcu_bh
    0 root        10     2  0.0 migration/0
    0 root        11     2  0.0 watchdog/0
    0 root        12     2  0.0 cpuhp/0
    0 root        13     2  0.0 cpuhp/1
    0 root        14     2  0.0 watchdog/1
    0 root        15     2  0.0 migration/1
    0 root        16     2  0.0 ksoftirqd/1
    0 root        18     2  0.0 kworker/1:0H
    0 root        19     2  0.0 cpuhp/2
    0 root        20     2  0.0 watchdog/2
```

Modelo de Processos - Espaço de Endereçamento

- Área de memória **onde o programa será armazenado**, além do espaço para os dados utilizados por ele
- Cada processo possui seu próprio espaço de endereçamento, que deve ser protegido de outros processos

Modelo de Processos - Espaço de Endereçamento

- Espaço de Endereçamento possui:
  - **Segmento de Texto**: código executável do programa
  - **Dados**: variáveis
  - **Pilha de Execução**
    - Controla a execução do processo
    - Empilha chamadas de procedimentos (funções)

Criação de Processos

- Sistemas Operacionais precisam de **uma forma de criar processos**
  - Em sistemas simples, todos os processos podem ser instanciados quando o sistema for ligado
  - Em outros casos (mais genéricos), **são necessárias formas de se criar e terminar processos**

Criação de Processos

- Processos podem ser criados de **quatro formas** principais:

  a. **Inicialização do Sistema**

     - Durante a inicialização, diversos processos são criados
     - Podem ser divididos entre:
       - **Primeiro plano**: interagem com o usuário
       - **Segundo plano**: não interagem com o usuário (monitor de um serviço)
     - Processos de segundo plano são chamados ***daemons***

Criação de Processos

- Processos podem ser criados de **quatro formas** principais:

  b. Chamada de Sistema

  - **Feita por um processo em execução**
  - Utilizado quando um trabalho é formulado em processos relacionados
    - Funcionam de maneira independente
  - Em termos de velocidade, o multiprocessamento se aproveita dessa possibilidade
    - Permite processos independentes executarem em diferentes CPUs

Criação de Processos

- Processos podem ser criados de **quatro formas** principais:

  c. **Um requisição do usuário para criar um novo processo**

     - Em sistema interativos, digitando um comando ou clicando em um ícone

  d. **Início de uma tarefa em lote**

     - Em computadores de grande porte, quando o mesmo possui recursos insuficientes para a execução da tarefa
     - Processamento de cheques, em um banco, ao final do dia

Criação de Processos

- Para todos os casos descritos, **um processo é o responsável por criar outro processo**
  - Uma chamada de sistema é a responsável pela criação
  - No UNIX, a chamada responsável por criar processos é a **`fork`**
    - **Cria um clone exato do processo que a chamou**
    - Após a **`fork`**, esses processos possuem a mesma imagem de memória, mesmas variáveis de ambiente e os mesmos arquivos abertos
    - Dessa forma, o processo filho executa outra chamada de sistema para executar um novo programa (**`execve`**), se for o caso

Criação de Processos

- Após um processo ser criado, pai e filho possuem seus próprios espaços de endereços
  - **Mudanças posteriores não são visíveis entre processos**
  - Existem casos onde não é interessante gerar a cópia instantânea do espaço de endereços
    - **Copy-on-write**: somente quando o pai ou filho tentam escrever na memória é que a separação, de fato, ocorre

Término de Processos

- Cedo ou tarde, um processo termina a sua execução
- O término pode ocorrer por **quatro condições** distintas:

a. Saída Normal (voluntária)

- Quando um processo termina o seu trabalho
- Há uma chamada (`exit`) para informar o término ao SO

b. Saída por Erro (voluntária)

- Processo descobre um erro fatal
- Exemplo: compilação de um arquivo que não existe

Término de Processos

- Condições que terminam um processo:

c. Erro fatal (involuntária)

- Erro causado pelo processo (erro de programação)
- Exemplo: divisão por zero, referência a endereço de memória inexistente

d. Cancelamento por outro processo (involuntária)

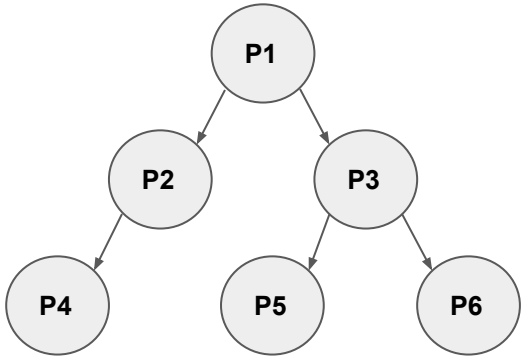
- Um processo executa uma chamada de sistema dizendo ao SO para cancelar algum outro processo
- Exemplo: chamada `kill` (Unix)
  - Precisa ter autorização para tal

Tipos de Processos

- Processos são classificados de acordo com o tipo de processamento que eles realizam.
- Existem dois tipos de processos:
  a. **CPU-bound**: Passa a maior parte do tempo utilizando a CPU
  b. **I/O-bound**: Passa a maior parte do tempo realizando operações de entrada e saída.

Hierarquia de Processos

- Quando um processo cria outro, há uma associação entre esses processos (pai e filho).
  - Um processo filho pode criar outros processos.
  - Forma-se, então, uma hierarquia de processos.



Hierarquia de Processos

- Durante a inicialização do UNIX, um processo chamado ***init*** é iniciado
  - Bifurca-se em diversos terminais
    - Um processo novo para cada terminal
  - Terminal é um processo que espera conexões
    - Dada uma conexão, um `shell` é executado para receber comandos
    - **Comandos iniciam novos processos**
- Assim, **todos os processos** do sistema **pertencem** a uma única **árvore de hierarquia** de processos
- No Windows não há o conceito de hierarquia de processos

Estados de Processos

- Apesar de independentes, diversos casos, processos precisam interagir
  - A saída de um processo pode ser utilizada com entrada de outro
    - Exemplo: `cat ~/.bash_history | grep make`
  - Como consequência, um processo pode não ter a entrada logo de início, pois o outro processo está processando o que será repassado
  - Assim, essa dependência exige uma **forma do processo ficar bloqueado** ou esperando o que é necessário para que ele faça os seus processamentos

Estados de Processos

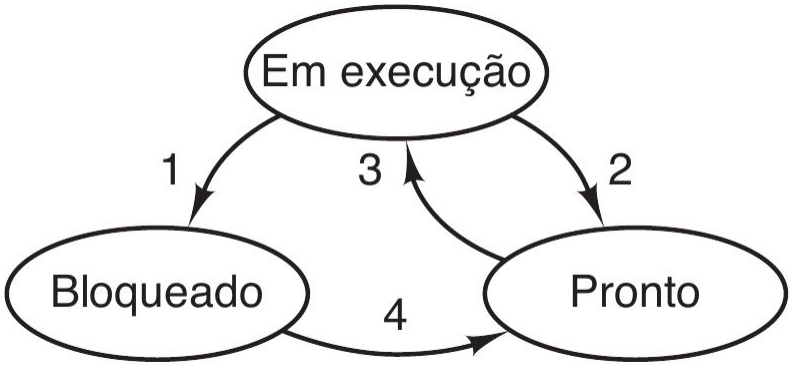
- Quando um processo está bloqueado, ele o está porque:
  - Não pode, por algum motivo, continuar executando:
    - Pode estar **aguardando recurso**
    - Ou a **CPU está alocada a outro processo** naquele momento
  - Ambos os casos são completamente diferentes:
    - Aguardo de recurso é inerente ao problema em execução
    - Alocação de CPU é uma tecnicidade do sistema

Estados de Processos

- Assim, um processo pode assumir três estados:
  a. **Em Execução**: utilizando CPU naquele momento
  b. **Pronto**: temporariamente parado e outro processo está executando
  c. **Bloqueado**: incapaz de ser executado até a ocorrência de evento externo

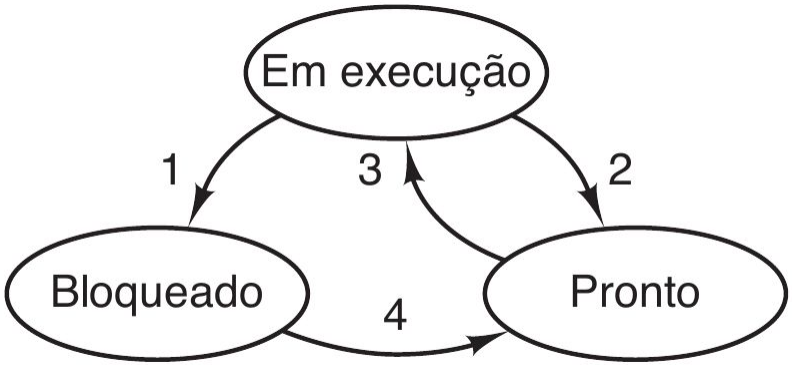
Estados de Processos

- Assim, um processo pode assumir três estados:
  a. **Em Execução**: utilizando CPU naquele momento
  b. **Pronto**: temporariamente parado e outro processo está executando
  c. **Bloqueado**: incapaz de ser executado até a ocorrência de evento externo

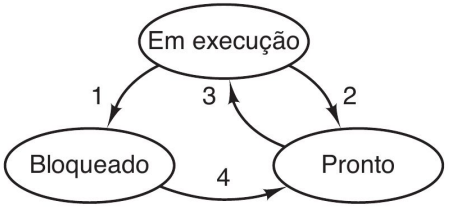


Estados de Processos

- Em Execução e Pronto são similares
  - O processo está disposto a ser executado
- Bloqueado, o processo não pode ser executado
  - Ainda que a CPU esteja ociosa



Estados de Processos - Transições



- Transição 1 ocorre quando o SO descobre que um processo não pode continuar
- Transição 2 e 3 são causadas pelo escalonador de processos
  - Transição 2: O processo foi executado por tempo suficiente
  - Transição 3: Divisão de parcela justa
- Transição 4 é causada quando o evento externo acontece
- **Escalonador de Processos tem a atividade organizacional de decidir**, dentre os processos prontos, o próximo a executar

Estados de Processos - Transições

- Usando esse modelo de estados, fica mais fácil entender o funcionamento dentro do SO
  - Processos executam programas que repassam saídas
  - Processos lidam com tarefas ou serviços
  - Em uma interrupção, o SO decide parar o processo atual, para executar um outro processo
    - O processo que **solicitou recurso vai para o estado bloqueado**

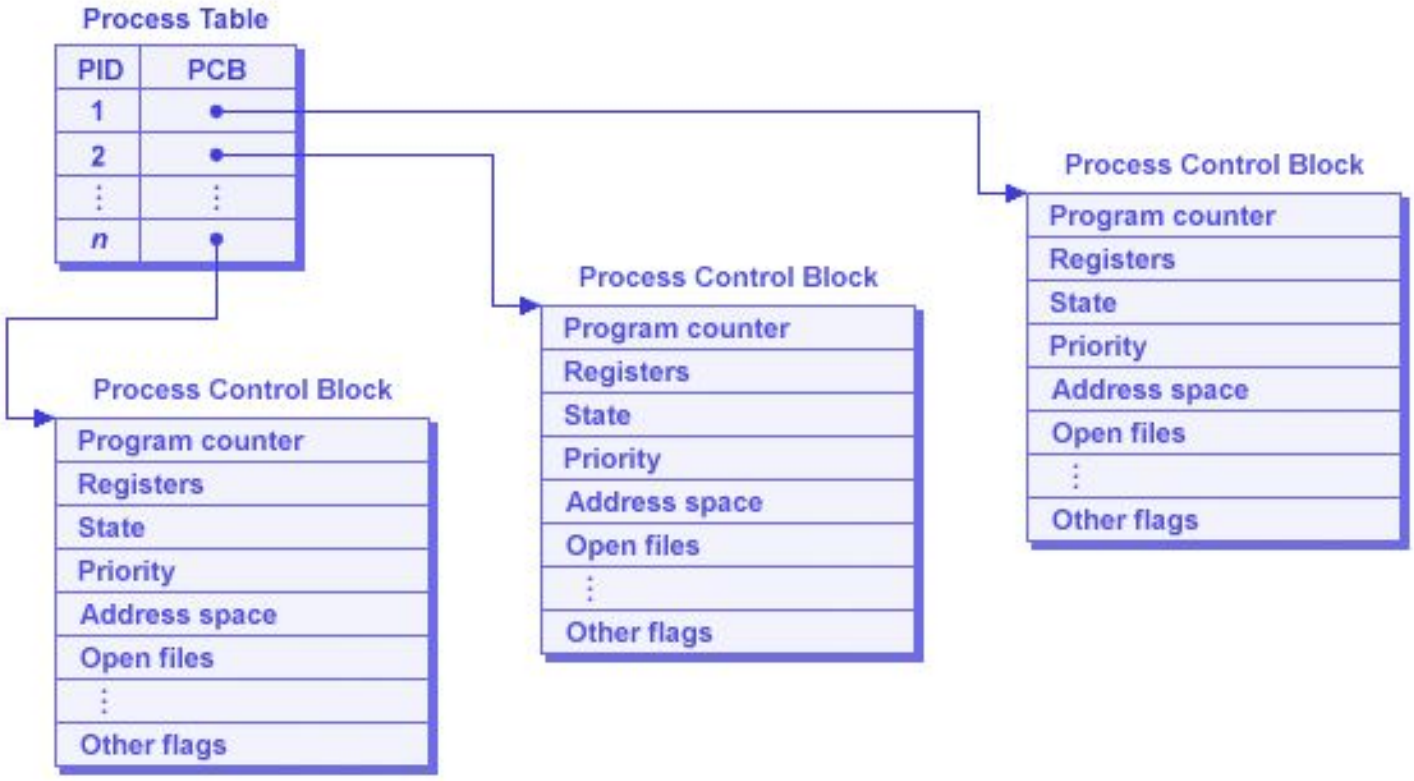
Implementação de Processos

- Para implementar o modelo, o SO mantém uma tabela chamada de

  Tabela de Processos (ou Bloco de Controle de Processo (BCP))

  - Cada processo possui uma entrada nessa tabela
- A tabela de processos tem campos que dizem respeito:
  - Á gerência do processo
  - Á gerência da memória
  - Á gerência de arquivos

Implementação de Processos



Implementação de Processos

- Os campos da tabela de processos variam de acordo com o SO.

Gerenciamendo De Processo	Gerenciamento de Memória	Gerenciamento de Arquivo
Registros	Ponteiro para informações sobre o segmento de texto	Diretório-raiz
Contador de Programa	Ponteiro para informações sobre o segmento de dados	Diretório de Trabalho
Palavra de Estado do Programa	Ponteiro para informações sobre o segmento de pilha	Deescritores de Arquivo
Ponteiro da Pilha		ID do Usuário
Estado do Processo		ID do Grupo
Prioridade		
Parâmetros de Escalonamento		
ID do Processo		
Processo Pai		
Grupo de Processo		
Sinais		
Momento em que o Processo foi Iniciado		
Tempo de CPU usado		
Tempo de CPU do processo filho		
Tempo do Alarme Seguinte		

Implementação de Processos

- A partir da tabela de processos, vem a ideia geral de múltiplos processos
  - Associada a cada classe de Entrada e Saída há um Vetor de Interrupção
    - Contém o endereço da rotina de serviço de interrupção.
  - Quando uma interrupção é acionada, a CPU salva o estado atual e executa o tratamento apontado pelo vetor
  - Garante que nenhuma das tarefas do SO entre em conflito

Implementação de Processos

- Passos quando ocorre uma **interrupção**:
  a. O hardware empilha o contador de programa
  b. O hardware carrega o novo PC, a partir do arranjo de interrupções
  c. O vetor de interrupções em linguagem de montagem salva os registradores
  d. A rotina em linguagem de montagem configura uma nova pilha
  e. O serviço de interrupção (em C) executa
  f. Escalonador decide qual o próximo processo é o próximo a executar
  g. A rotina em C retorna para o código em linguagem de montagem
  h. A rotina em linguagem de montagem inicia o novo processo atual

Modelagem da Multiprogramação

- Usando multiprogramação, a utilização da CPU pode ser aumentada
  - Com a multiprogramação, mais de um processo compete pela CPU, de forma que quando um processo estiver aguardando por algum evento, outro processo pode utilizar a CPU
    - Se um processo realiza computações em 20% do tempo, será que 5 processos são suficientes para manter a CPU ocupada o tempo todo?

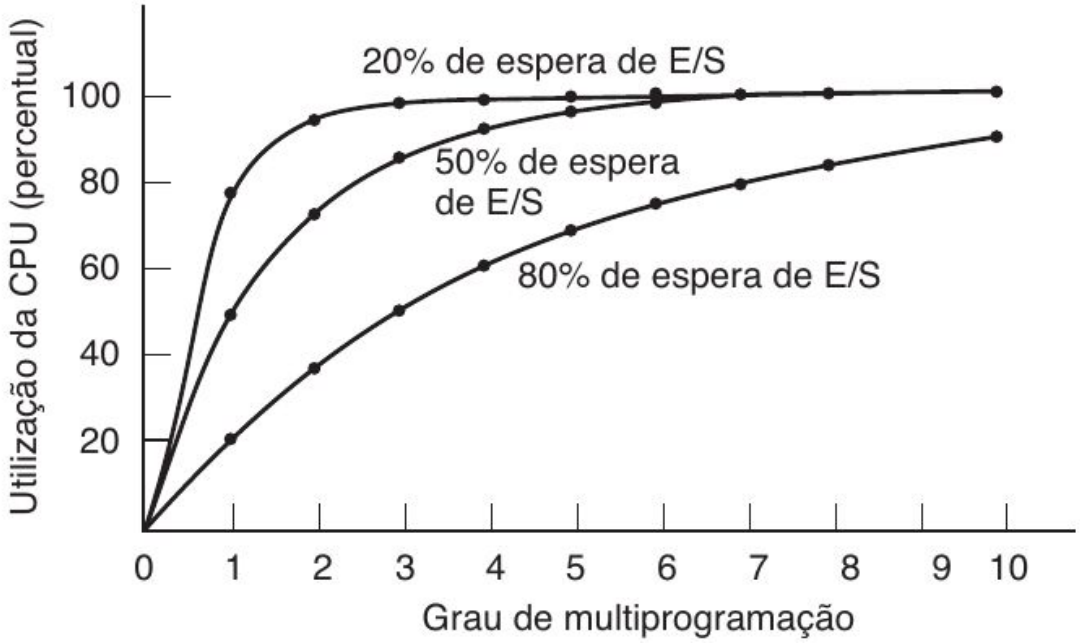
Modelagem da Multiprogramação

- Resultados melhores podem ser visualizados por uma abordagem probabilística
  - Um processo passa uma fração de tempo p esperando E/S
  - Com n processos na memória ao mesmo tempo, a probabilidade todos estarem esperando E/S ao mesmo tempo é de p^n
  - Assim, a utilização de CPU é dada por:

$$Utilização\ da\ CPU = 1 - p^n$$

Modelagem da Multiprogramação

- A partir da modelagem probabilística, é possível ter uma visualização do comportamento da CPU.



Modelagem da Multiprogramação

- Exercício para prática da Modelagem da Multiprogramação:
  - Tempo esperando por E/S - 80%
  - Quantidade de Processos na Memória (ao mesmo tempo) - 10

$$p = 0.8 \qquad n = 10$$

$$Utilização = 1 - p^n$$

$$Utilização = 1 - 0.8^{10}$$

$$Utilização = 1 - 0.10737 = 0.89 = 89\%$$

Modelagem da Multiprogramação

- Modelagem probabilística é uma aproximação
  - Assume que todos os processos são independentes
    - Não é verdade, pois há dependência entre processos
  - Curvas reais vão ser diferentes, mas a aproximação dá uma resposta aceitável
  - Permite o uso para previsões
    - Quanto de memória eu preciso para melhorar em X a Utilização?

Conclusão